Documentación del frontend — AlojaME

> Qué se construyó en el frontend Angular, cómo está organizado y por qué se tomó cada decisión de diseño. Complementa al `README.md` de la raíz (puesta en marcha y despliegue), a `frontend/README.md` (comandos y convenciones) y a *Bitacora_Tecnica_AlojaME.pdf* (errores encontrados y resueltos).

---

1. Qué es y para quién

AlojaME es una plataforma de alquileres temporarios en Mar del Plata. Tiene cuatro tipos de visitante, y cada uno ve una aplicación distinta:

Perfil	Qué puede hacer
Visitante (sin cuenta)	Navegar el catálogo: búsqueda con filtros, mapa, detalle del alojamiento, fotos y reseñas.
Cliente	Reservar con un calendario de fechas, pagar con MercadoPago, cambiar fechas, cancelar, dejar reseñas y ver sus gastos.
Anfitrión	Publicar y editar alojamientos (fotos, servicios, ubicación en el mapa), confirmar y finalizar reservas, cargar su alias de cobro y ver sus cobros e ingresos.
Administrador	Gestionar usuarios (con una ficha por usuario), los catálogos de tipos y servicios, todas las reservas, reembolsos manuales, el flujo de dinero y las estadísticas globales.

2. Cumplimiento de la consigna (Programación IV)

Requisito	Dónde se cumple
1. Íntegramente en Angular 22	`frontend/` — Angular 22.2, componentes standalone, sin NgModules.
2. Al menos dos CRUD sobre entidades distintas	Tipos de alojamiento y servicios (admin), alojamientos (anfitrión), usuarios (admin) y reservas (los tres roles). Detalle en `frontend/README.md`.
3. Login con distintos roles	`AuthService`: JWT del backend, rol decodificado del token. Roles: cliente, anfitrión y administrador.
4. Guards por rol	`authGuard`, `roleGuard([...])`, `soloVisitanteGuard` e `idNumericoGuard` en `app.routes.ts`.
5. Comunicación HTTP con el backend de Programación III	Todos los servicios de `core/services/` usan `HttpClient` contra la API Spring Boot real (`GA/`). No hay json-server ni datos simulados.
Funcionalidad elevada	Pagos reales (MercadoPago), mapa, calendario de reservas, cancelaciones con reembolso, baja en cascada, ficha de usuario, estadísticas.

Requisito	Dónde se cumple
Diseño visual destacado	Identidad propia (paleta, logotipo y mascota "Kaza"), carta de avisos animada, gráficos con Chart.js. Ver sección 6.
Completamente responsive	Diseño mobile-first verificado en 360, 768, 1024 y 1280 px. Ver sección 7.
Documentación y README	Este documento, `README.md`, `frontend/README.md` y *Bitacora_Tecnica_AlojaME.pdf*.

3. Arquitectura

```
src/app/
├── core/            lo que no se ve: configuración, guards, interceptor,
│   │                modelos, servicios HTTP y utilidades
│   ├── services/    un servicio por recurso de la API (+ AuthService,
│   │                ConfirmacionService, BajaUsuarioService)
│   └── utils/       fechas locales, mensajes de error, reglas de cancelación…
├── shared/          piezas visuales reusables (carta de avisos, calendario,
│                    gráfico de barras, mapa, visor de fotos, Kaza…)
├── layout/          encabezado, pie, páginas 403 y 404
└── features/        una carpeta por perfil: public, auth, cliente,
                     anfitrion, admin (una pantalla por carpeta)
```

- **Componentes standalone + signals.** Cada pantalla guarda su estado en `signal()` y deriva el resto con `computed()`. Es la forma actual de Angular, evita suscripciones manuales y hace que la vista se actualice sola.
- **Un servicio por recurso.** Las pantallas nunca llaman a `HttpClient` directo: piden datos a `ReservaService`, `AlojamientoService`, etc. Si cambia un endpoint, se toca un solo archivo.
- **Carga diferida.** Casi todas las pantallas se descargan recién cuando se visitan (`loadComponent`). Chart.js y Leaflet (el mapa) también se cargan a pedido. La carga inicial queda por debajo de 500 kB.
- **URL de la API fuera del código.** Se lee de `public/env.js`, así el mismo build sirve en desarrollo (`http://localhost:8081`) y en el servidor (`/api`, detrás de Nginx) sin recompilar.

4. Autenticación y protección de rutas

1. **Login.** `POST /auth/login` devuelve un JWT. `AuthService` lo guarda, decodifica el rol y lo expone como signals (`estaLogueado()`, `rol()`, `usuario()`).
2. **Cada petición.** `authInterceptor` agrega `Authorization: Bearer …`.
3. **Sesión vencida o cuenta desactivada.** El token tiene vencimiento: un temporizador cierra la sesión justo a esa hora. Si el backend responde 401, o 403 con un token que ya no sirve, se verifica la sesión con `/Usuario/me` y, si corresponde, se cierra con un aviso. Así un usuario desactivado por el admin queda afuera de inmediato.
4. **Guards.**
   - `authGuard`: exige sesión; si no hay, lleva a `/login` y después vuelve.
   - `roleGuard(['ADMIN'])`: exige un rol; si no coincide, muestra `/403`.
   - `soloVisitanteGuard`: quien ya inició sesión no ve login ni registro.

    - `idNumericoGuard`: `/alojamientos/abc` va a "no encontrada" en vez de quedarse cargando.
5. **Cambio de email.** El email es la identidad dentro del JWT. Si el usuario lo cambia, el backend devuelve un token nuevo en el encabezado `X-New-Token` y el frontend lo reemplaza sin cortar la sesión.

Además de los guards, las pantallas de edición verifican que el recurso sea del usuario: un anfitrión no puede editar el alojamiento ni la galería de otro cambiando el número en la URL (`core/utils/duenio.util.ts`). El backend, por su parte, rechaza cualquier operación sobre recursos ajenos.

5. Decisiones de diseño (y por qué)

Búsqueda y filtros del lado del cliente

El backend devuelve el catálogo completo y no acepta filtros por parámetros. En lugar de modificarlo, la búsqueda filtra en el navegador (texto sin tildes ni mayúsculas, precio, huéspedes y servicios) y pagina de a 12 resultados. Para filtrar por servicio se consultan una sola vez, en paralelo, los servicios de cada alojamiento. Los servicios se eligen en una carta aparte con buscador, porque el catálogo puede ser largo.

Calendario de rango para reservar

Las fechas se eligen en un calendario propio (`shared/calendario-rango`): check-in y check-out con los días intermedios marcados y los días ocupados o pasados en gris. Evita el error más común (elegir fechas ya reservadas) antes de llegar al backend. El mismo componente sirve para elegir períodos en el flujo de dinero.

Fechas siempre en hora local

`toISOString()` convierte a UTC y en Argentina (UTC-3) puede correr la fecha un día. Todas las fechas pasan por `core/utils/fecha.util.ts`, que trabaja en hora local.

Una sola carta para confirmaciones y avisos

No se usan `alert()` ni `confirm()` del navegador. Toda confirmación ("¿Cancelar esta reserva?") y todo resultado ("Se te devuelven $X") se muestra en la misma carta (`shared/confirmacion`, vía `ConfirmacionService`), con Kaza como sello en la esquina. Antes de una acción con dinero, la carta explica qué va a pasar con la plata; después, qué pasó. El foco arranca en "Volver" para que un Enter apurado nunca ejecute la acción peligrosa.

Reglas de cancelación explicadas en pantalla

`core/utils/cancelacion.util.ts` replica las reglas del backend (reembolso total, retención del 35 % si el cliente cancela con menos de 48 h, etc.) solo para **mostrar** el resultado antes de confirmar. El cálculo real lo sigue haciendo el backend.

Pagos simulados para la demostración

MercadoPago de prueba no permite reembolsos ni liquidaciones a anfitriones. Con `PAGOS_SIMULADOS=true` en el backend, esos dos movimientos se registran sin llamar a MercadoPago, y las reservas cambian de estado igual. El frontend lo sabe por `env.js` (`pagosSimulados: true`) y lo aclara en la carta: "[Transacción simulada…]". El cobro al cliente sigue siendo real.

Baja de usuarios en cascada

El backend, al desactivar una cuenta, solo cambia su estado. Para que no queden alojamientos publicados de un anfitrión inactivo ni solicitudes de un cliente que ya no puede pagar, `BajaUsuarioService` primero muestra qué se va a ver afectado y, al confirmar, pausa los alojamientos y cancela las reservas sin pago. Las reservas ya pagadas no se tocan solas (cancelarlas devuelve dinero): quedan para que el administrador decida desde la ficha del usuario.

Ficha de usuario para el administrador

Desde Gestión de usuarios, el nombre abre la ficha: números, alojamientos a su nombre (publicados o pausados) y reservas, con links a cada alojamiento, cliente y anfitrión. En "Todas las reservas" y en el detalle de cada reserva también son links. El administrador puede seguir cualquier recorrido sin buscar a mano.

Mensajes de error en lenguaje claro

El backend responde errores en dos formatos y en lenguaje técnico. `http-error.util.ts` los normaliza y traduce los más comunes ("Esas fechas ya están reservadas. Probá con otras.", "El usuario o la contraseña no son correctos.").

Sin librerías de componentes

Se usa CSS propio, sin Angular Material, Bootstrap ni Tailwind. Así la identidad visual es propia y el peso de la app es menor. Los únicos agregados son Chart.js (gráficos) y Leaflet (mapa).

6. Diseño visual

- **Paleta:** tinta casi negra, tomate, dorado, azul y acero, definidos una sola vez como variables en `src/styles.css`. Ninguna pantalla usa colores sueltos.
- **Tipografía:** Plus Jakarta Sans para el texto y Jaybird para el logotipo. Jaybird es gratuita solo para uso personal; para un uso comercial hay que comprar la licencia.
- **Kaza:** la mascota (una casita con cara). Aparece en el logo, en el avatar del usuario, en las páginas 403 y 404, y como insignia de la carta de avisos. En la carta entra con un "pop" después de la tarjeta y se balancea sobre el centro de su cara.
- **Gráficos (Chart.js):** pensados para quien no sabe leer gráficos. Arriba de cada uno hay una frase con lo importante ("X encabeza con $Y, el 40 % del total"), cada barra tiene su valor escrito y la más alta va resaltada. El botón "i" junto al título abre una explicación de Kaza sobre qué muestra el gráfico y cómo se calcula. Debajo, "Ver los datos" muestra los mismos números en una tabla.

- **Estados vacíos y de carga:** esqueletos mientras se carga y mensajes con una acción cuando no hay resultados ("Limpiar filtros", "Publicar alojamiento").

7. Responsive

Mobile-first con tres cortes: base para celular (probado en 360 px), tablet desde 768 px y desktop desde 1024 px. Las utilidades globales de `src/styles.css` resuelven los casos repetidos:

- `.form-grid`: formularios en una columna en celular y en dos desde tablet.
- `.table-cards` con `data-label`: las tablas se convierten en tarjetas debajo de 1024 px.
- `.kpi-grid`: tarjetas de números que se reacomodan solas.

Todas las pantallas se verificaron con los cuatro perfiles en 360, 768, 1024 y 1280 px: sin scroll horizontal y con controles de al menos 44 px de alto.

8. Limitaciones conocidas

- **Liquidaciones de MercadoPago:** la API de liquidaciones rechaza las credenciales de prueba. Para la demostración se usa el modo de pagos simulados (sección 5).
- **Funciones que necesitan cambios en el backend:** búsqueda por disponibilidad de fechas, recuperación de contraseña, límite de huéspedes por reserva y emails de reservas. Quedaron listadas en *Pendientes_Backend_AlojaME.pdf*.
- **Tests automatizados:** el frontend no tiene tests automatizados. Se verificó con recorridos completos por rol contra el backend real.

9. Cómo probarlo

1. Levantar backend y frontend como indica el `README.md` de la raíz.
2. Cargar los datos de demostración: `GA/db/datos_demo_completa.sql`. Al terminar, el script imprime una guía con cada cuenta de prueba y qué caso muestra (reserva por pagar, cancelación tardía, reserva para finalizar, usuario desactivado, etc.).
3. Para la demostración, activar `PAGOS_SIMULADOS=true` en el backend y `pagosSimulados: true` en `env.js`.